EJERCICIO DE ENTRADA Y SALIDA DE DATOS CON POLIMORFISMO.

Ejercicio 1. Juego de Animales.

Vamos a realizar un juego sobre animales con perros y gatos.

- De cada uno se guardará su **nombre** y su **edad**.
- En el caso de los perros, se quiere saber también **la longitud del rabo**: largo o no, y de los gatos, **la del bigote**: largo o no.
- También guardaremos **cómo hablan** cada uno. Los gatos: "MIAU", los perros: "GUAU".
- Para saber el tipo de animal, perro o gato, usaremos un método **getClase().**

Para guardar la información anterior, crearemos una clase **GestionarAnimales**.
En ella se podrá guardar información sobre cinco perros y cinco gatos con uso de arrays de tamaño 5.

- Con un método **addPerro(Perro perro)** que permitirá añadir un perro.
    - Devolverá un boolean indicando si se añadió correctamente el perro.
    - Si intentamos añadir un nuevo perro y el array está ocupado, devolverá false.
- Lo mismo para el caso de los gatos: método **addGato(Gato gato).**
- Dispondrá de un método **WriteAnimalsDisk(String fichero)** para guardar los objetos que se encuentren en los dos arrays (perros / gatos).
    - Escribiremos en el fichero, antes de los objetos, el número total a escribir.
- Dispondrá de un método **ReadAnimalsDisk(String fichero)** que leerá de disco los objetos guardados del paso anterior.
    - Primero leeremos el número de animales guardados en disco para dar memoria al array que debe devolver el método.
    - Después leeremos todos los animales que se irán guardando en el array que tenemos que devolver.
    - Se actualizarán los datos de la clase (los arrays que guardan los gatos y los perros).

Crea una clase **ProbarAnimales** con un método main que haga uso de la clase anterior.

Ejercicio 2 : Gestión de vehículos (Coches y Motos)

El objetivo es desarrollar un programa en **Java** que permita gestionar una flota de **coches y motos** con las siguientes características:

1. **Coches** y **Motos** heredan de una clase base Vehiculo:
    - **Todos los vehículos** tienen marca, modelo y año de fabricación.
    - **Los coches** tienen un atributo adicional esElectrico (true si es eléctrico).
    - **Las motos** tienen el atributo cilindrada (su potencia en **cc**).
    - Cada vehículo tiene un método getTipoVehiculo() que indica si es un **Coche** o una **Moto**.
2. **Gestión de Vehículos en GestionarVehiculos**:
    - Se pueden **almacenar hasta 5 coches y 5 motos**.
    - Métodos para **agregar coches y motos** a la flota.
    - Métodos para **guardar y recuperar vehículos desde un archivo** mediante **serialización**.
    - Método para **mostrar la lista de vehículos almacenados**.
3. **Prueba del programa en ProbarVehiculos**:
    - Se agregan vehículos a la gestión.
    - Se guardan en un archivo binario vehiculos.dat.
    - Se cargan desde el archivo y se muestra el resultado.

Ejercicio 3: Gestión de Empleados con Acceso Aleatorio a un Archivo Binario (Evaluable)

Se debe desarrollar un programa en **Java** que gestione empleados de **dos tipos: Asalariados y Por Horas**, almacenándolos en un **archivo binario** con **acceso aleatorio** (RandomAccessFile).

Estructura del Archivo Binario (empleados.dat)

Cada **empleado** ocupará exactamente **40 bytes** en el archivo y se almacenará en la siguiente estructura:

Posición en el archivo	Tamaño (bytes)	Tipo de dato	Descripción
0	4	int	**ID del empleado**
4	Variable	UTF-8	**Nombre**
24	4	int	**Edad del empleado**
28	8	double	**Salario del empleado**
36	4	int	**Tipo (1 = Asalariado, 2 = Por Horas)**

Cada **nuevo empleado** se escribirá en una **posición aleatoria** del archivo según su ID.

Requisitos del Programa

1. **Clases Empleado, EmpleadoAsalariado y EmpleadoPorHoras**
   - Clase base Empleado con **ID, nombre, edad y salario base**.
   - Subclases:
     - EmpleadoAsalariado: Su salario es fijo.
     - EmpleadoPorHoras: Su salario depende de **horas trabajadas** y **tarifa por hora**.
   - Implementar **una interfaz Calculable** con el método calcularSalario().
   - **Sobrecarga y sobreescritura** en constructores y métodos.
2. **Clase GestionEmpleados**
   - Método agregarEmpleado(Empleado e): Guarda un **nuevo empleado** en el archivo binario en la **posición** ID * 40 (cada registro ocupa 40 bytes).
   - Método leerEmpleado(int id): Lee un **empleado específico** desde el archivo en la **posición** ID * 40.
   - Método modificarSalario(int id, double nuevoSalario): Modifica el **salario** de un empleado accediendo a la posición ID * 40 + 28.
   - Método listarEmpleados(): Recorre el archivo y muestra todos los empleados almacenados.
3. **Clase ProbarEmpleados**
   - Agrega **4 empleados** (2 asalariados y 2 por horas).
   - Guarda los empleados en el archivo.
   - Muestra todos los empleados guardados.
   - Modifica el salario de un empleado accediendo a la posición exacta en el archivo.
   - Vuelve a listar los empleados para comprobar el cambio.